

Practical Work 5: The Longest Path

Nguyen Trong Bach
Student ID: 23BI14057

December 6, 2025

1 Introduction

This practical implements a small MapReduce style project called *The Longest Path*. The goal is to analyse a set of text files where each line contains one full file path (for example, the output of the command `find /`) and to find the longest path or paths. Instead of using a full MapReduce framework, I design a simple MapReduce-like program in C++.

2 Input and Output

- **Input:** a set of text files, each containing one full file path per line. Each file can be considered as the data coming from one laptop.
- **Output:** the maximum path length and all paths whose length is equal to that maximum.

3 Mapper and Reducer Design

Mapper

Each input file is processed by a logical Mapper, implemented as the function `map_file(const std::string&)`. For a given file the Mapper:

1. reads the file line by line;
2. for each non-empty line computes the length of the path (number of characters);
3. keeps track of the current maximum length inside that file;
4. stores all paths whose length is equal to this local maximum.

The Mapper returns a structure `MapResult` containing the local maximum length and the list of paths that achieve this length.

Reducer

The Reducer is implemented by the function `reduce_result(const MapResult&, std::size_t&, std::vector<std::string>&)`. It maintains two global variables: the global maximum length and the list of all paths that achieve this maximum. For each `MapResult` received from a Mapper the Reducer:

- replaces the global result if the local maximum is larger;
- appends the local paths if the local maximum is equal to the global maximum;
- ignores the local result if it is smaller.

This behaviour is analogous to MapReduce, where all partial results for the same key are combined in the Reduce phase.

4 System Organization

The whole system is implemented in a single source file `longest_path.cpp`. The main program follows these steps:

1. read the list of input file names from the command line;
2. for each file, call the Mapper `map_file` to obtain a `MapResult`;
3. call the Reducer `reduce_result` to combine all `MapResult` objects into a global result;
4. after all files have been processed, print the global maximum path length and the corresponding paths.

5 Implementation Snippet

An example excerpt from the Mapper function is shown below:

```
MapResult map_file(const std::string &filename) {
    MapResult result;
    std::ifstream in(filename);
    if (!in) {
        std::cerr << "Cannot_open_input_file:_ " << filename << "\n";
        return result;
    }

    std::string line;
    while (std::getline(in, line)) {
        std::size_t len = line.size();
        if (len == 0) continue;

        if (len > result.max_len) {
            result.max_len = len;
            result.paths.clear();
            result.paths.push_back(line);
        } else if (len == result.max_len) {
            result.paths.push_back(line);
        }
    }
    return result;
}
```

6 Group Work

This practical work was completed individually. I was responsible for designing the Mapper and Reducer logic, implementing the C++ program and writing this report.

7 Conclusion

The Longest Path practical shows how the MapReduce idea can be applied to a different problem than Word Count. By clearly separating the Mapper and Reducer stages, the code is easy to understand and could later be extended to run in parallel using multiple processes or machines.